Overview of GCN

Semi-supervised classification with graph convolutional networks

Kanglei Zhou

August 7, 2020

Outline

1. Clarification
2. Background
3. Methods
4. Experiments
5. Discussion
6. References

Outline

1. Clarification
2. Background
3. Methods
4. Experiments
5. Discussion
6. References

Graph Fourier Transform

Let $G = (V, E)$ be a weighted graph, $\mathbf{L}$ be its corresponding graph Laplacian, and $f : V \to \mathbb{R}$ is a funciton defined on the vertices of G.

Definition (Graph Fourier Transform)

The Graph Fourier Transform of f is defined as

$$\mathcal{GF}[f](\lambda_l) = \hat{f}(\lambda_l) =< \boldsymbol{f}, \boldsymbol{u}_l >= \sum_{i=1}^{n} f(i) u_l^T(i). \tag{1}$$

which is equal to

$$\hat{\boldsymbol{f}} = \mathbf{U}^T \boldsymbol{f}.$$

Spectral & Spatial

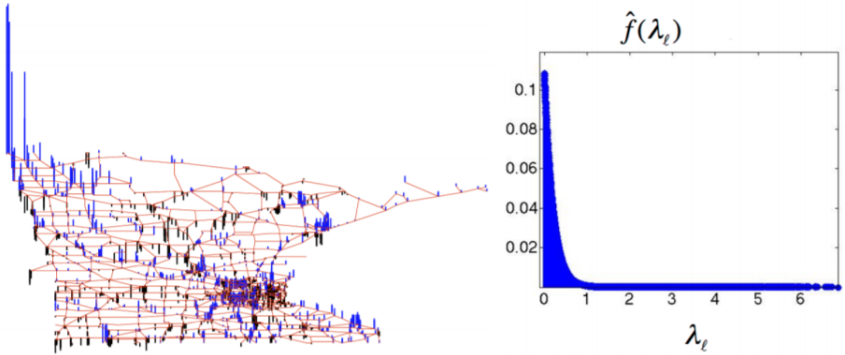


Figure 1: The graph Fourier transform (and its inverse) gives a way to represent a signal in two different domains: the vertex domain and the graph spectral domain.

About structured data

In this thesis, Deep Learning with Graph-Structured Representations, we propose novel approaches to machine learning with structured data. Our proposed methods are largely based on the theme of structuring the representations and computations of neural network-based models in the form of a graph, which allows for improved generalization when learning from data with both explicit and implicit modular structure.

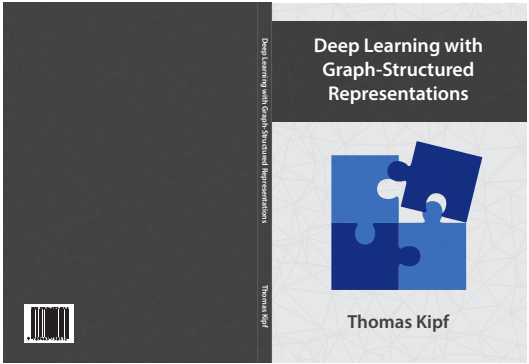


Figure 2: Thomas Kipf [1]. Download

Deep Learning with Graph-Structured Representations

How can we formalize this structure? A natural way to represent information in structured form is as a graph.

Definition

A graph is a data structure describing a collection of entities, represented as nodes, and their pairwise relationships, represented as edges.

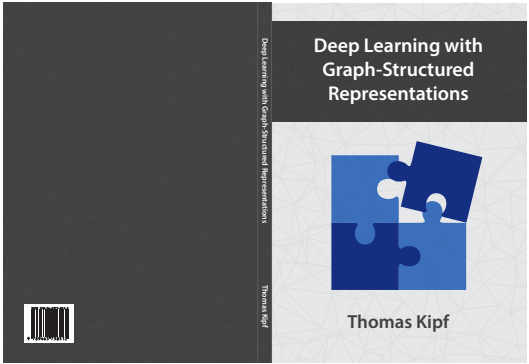
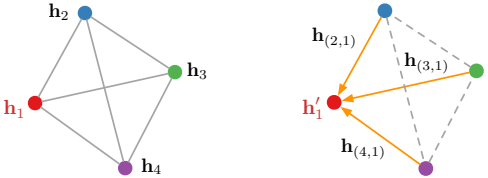


Figure 2: Thomas Kipf [1]. Download

Outline

1. Clarification
2. Background
3. Methods
4. Experiments
5. Discussion
6. References

Message passing update



(a) Node representations. **(b)** Message passing step.

Figure 2.1: Illustration of the message passing update in a graph neural network (GNN) on a fully-connected graph with four nodes. Each node is assigned a node representation $\mathbf{h}_i$ (left). In the message passing step (right), intermediate edge representations $\mathbf{h}_{(i,j)}$ are obtained from neighboring node representations $\mathbf{h}_i$, their initial node features $\mathbf{x}_i$ and edge features (if present) $\mathbf{x}_{(i,j)}$. Edge representations for incoming edges are aggregated to obtain updated node representations $\mathbf{h}'_i$.

$$\mathbf{h}_{i,j} = f_{\text{edge}}\left(\mathbf{h}_i, \mathbf{h}_j, \mathbf{x}_{(i,j)}\right),$$

$$\mathbf{h}' = f_{\text{node}}\left(\mathbf{h}_i, \sum_{j \in \mathcal{N}_i} \mathbf{h}_{(j,i)}, \mathbf{x}_i\right).$$

Latent variable model

We can understand a latent variable model as a probabilistic model that explains a set of observed variables $\mathbf{x}$ with a set of latent variables $\mathbf{z}$:

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p(\mathbf{z}) p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}) d\mathbf{z}$$

Graph-Based Semi-Supervised Learning

In graph-based semi-supervised learning, the graph structure is utilized in addition to both unlabeled and labeled data points, which take the role of nodes in the graph.

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda \mathcal{L}_{\text{reg}}. \tag{2}$$

where the original supervised loss $\mathcal{L}_{\text{sup}}$ only considers individual labeled nodes in isolation.

$$\mathcal{L}_{\text{reg}} = \sum_{i,j} A_{i,j} \left\| f(\mathbf{x}_i) - f(\mathbf{x}_j) \right\|^2 \propto f(\mathbf{X}) \boldsymbol{\Delta} f(\mathbf{X}), \tag{3}$$

where it relies on the assumption that connected nodes in the graph are likely to share the same label.

Spectral Graph Convolutions

A spectral convolution on a graph can be understood as a parameterized filtering operation that takes into account both node features (in this setting often described as signal) and the structure of a graph.

$$g_\theta * \mathbf{x} = \mathbf{U} g_\theta \mathbf{U}^T \mathbf{x}. \tag{4}$$

Spectral Graph Convolutions

A spectral convolution on a graph can be understood as a parameterized filtering operation that takes into account both node features (in this setting often described as signal) and the structure of a graph.

$$g_\theta * \mathbf{x} = \mathbf{U} g_\theta \mathbf{U}^T \mathbf{x}. \tag{4}$$

1. Multiplication with the eigenvector matrix $\mathbf{U}$ is $\mathcal{O}(N^2)$.

Spectral Graph Convolutions

A spectral convolution on a graph can be understood as a parameterized filtering operation that takes into account both node features (in this setting often described as signal) and the structure of a graph.

$$g_\theta * \mathbf{x} = \mathbf{U} g_\theta \mathbf{U}^T \mathbf{x}. \tag{4}$$

1. Multiplication with the eigenvector matrix $\mathbf{U}$ is $\mathcal{O}(N^2)$.
2. Computing the eigendecomposition of $\mathbf{L}$ might be prohibitively expensive for large graph.

Spectral Graph Convolutions

To circumvent this problem one can approximate $g_{\boldsymbol{\theta}}(\boldsymbol{\Lambda})$ by a truncated polynomial expansion, in terms of Chebyshev polynomials $T_k(x)$ to Kth order:

$$g_{\boldsymbol{\theta}'}(\boldsymbol{\Lambda}) \approx \sum_{k=0}^{K} \theta'_k T_k(\tilde{\boldsymbol{\Lambda}}), \tag{5}$$

where a rescaled $\tilde{\boldsymbol{\Lambda}} = \frac{2}{\lambda_{\max}}\boldsymbol{\Lambda} - \mathbf{I}_N$.

Chebyshev polynomials

The Chebyshev polynomials are recursively defined as

$$T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x), \tag{6}$$

where

$$\begin{cases} T_0(x) = 1 \\ T_1(x) = x \end{cases}.$$

Spectral Graph Convolutions

Therefore, a convolution of a signal $\mathbf{x}$ with a filter $g_{\boldsymbol{\theta}'}$ can be rewritten as

$$g_{\boldsymbol{\theta}'} * \mathbf{x} \approx \sum_{k=0}^{K} \theta'_k T_k(\tilde{\mathbf{L}})\mathbf{x} \tag{7}$$

where $\tilde{\mathbf{L}} = \frac{2}{\lambda_{\max}}\mathbf{L} - \mathbf{I}_N$ and $(\mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^T)^k = \mathbf{U}\boldsymbol{\Lambda}^k\mathbf{U}^T$.

Spectral Graph Convolutions

Therefore, a convolution of a signal $\mathbf{x}$ with a filter $g_{\boldsymbol{\theta}'}$ can be rewritten as

$$g_{\boldsymbol{\theta}'} * \mathbf{x} \approx \sum_{k=0}^{K} \theta'_k T_k(\tilde{\mathbf{L}})\mathbf{x} \tag{7}$$

where $\tilde{\mathbf{L}} = \frac{2}{\lambda_{\max}}\mathbf{L} - \mathbf{I}_N$ and $(\mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^T)^k = \mathbf{U}\boldsymbol{\Lambda}^k\mathbf{U}^T$.

1. Note that this expression is now K-localized since it is a Kth order polynomial in the Laplacian, i.e., it depends only on nodes that are at maximum K steps away from the central node (Kth order neighborhood), and hence it can be seen as a **spatial graph filter**.

Spectral Graph Convolutions

Therefore, a convolution of a signal $\mathbf{x}$ with a filter $g_{\boldsymbol{\theta}'}$ can be rewritten as

$$g_{\boldsymbol{\theta}'} * \mathbf{x} \approx \sum_{k=0}^{K} \theta'_k T_k(\tilde{\mathbf{L}})\mathbf{x} \tag{7}$$

where $\tilde{\mathbf{L}} = \frac{2}{\lambda_{\max}}\mathbf{L} - \mathbf{I}_N$ and $(\mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^T)^k = \mathbf{U}\boldsymbol{\Lambda}^k\mathbf{U}^T$.

1. Note that this expression is now K-localized since it is a Kth order polynomial in the Laplacian, i.e., it depends only on nodes that are at maximum K steps away from the central node (Kth order neighborhood), and hence it can be seen as a **spatial graph filter**.
2. The complexity of evaluating K-localized convolution is $\mathcal{O}(|\mathcal{E}|)$.

Outline

1. Clarification
2. Background
3. Methods
4. Experiments
5. Discussion
6. References

Graph Convolutional Networks

A single message passing step in the GCN model takes the following form:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\left(\mathbf{W}_0^{(l)}\right)^T \mathbf{h}_i^{(l)} + \sum_{j\in\mathcal{N}_i} c_{i,j}\left(\mathbf{W}_1^{(l)}\right)^T \mathbf{h}_j^{(l)}\right), \tag{8}$$

1. $\mathbf{h}_i^l \in \mathbb{R}^{d_l}$ is the hidden representation vector of node $i \in \mathcal{V}$ with dimensionality d_l after the lth message passing step (or 'layer')

Graph Convolutional Networks

A single message passing step in the GCN model takes the following form:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\left(\mathbf{W}_0^{(l)}\right)^T \mathbf{h}_i^{(l)} + \sum_{j\in\mathcal{N}_i} c_{i,j}\left(\mathbf{W}_1^{(l)}\right)^T \mathbf{h}_j^{(l)}\right), \tag{8}$$

1. $\mathbf{h}_i^l \in \mathbb{R}^{d_l}$ is the hidden representation vector of node $i \in \mathcal{V}$ with dimensionality d_l after the lth message passing step (or 'layer')
2. σ is a pointwise non-linearity, e.g., ReLU activation function

Graph Convolutional Networks

A single message passing step in the GCN model takes the following form:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\left(\mathbf{W}_0^{(l)}\right)^T \mathbf{h}_i^{(l)} + \sum_{j\in\mathcal{N}_i} c_{i,j}\left(\mathbf{W}_1^{(l)}\right)^T \mathbf{h}_j^{(l)}\right), \tag{8}$$

1. $\mathbf{h}_i^l \in \mathbb{R}^{d_l}$ is the hidden representation vector of node $i \in \mathcal{V}$ with dimensionality d_l after the lth message passing step (or 'layer')
2. σ is a pointwise non-linearity, e.g., ReLU activation function
3. $\mathbf{W}_0^{(l)}, \mathbf{W}_1^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ are learnable parameter matrices

Graph Convolutional Networks

A single message passing step in the GCN model takes the following form:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\left(\mathbf{W}_0^{(l)}\right)^T \mathbf{h}_i^{(l)} + \sum_{j\in\mathcal{N}_i} c_{i,j}\left(\mathbf{W}_1^{(l)}\right)^T \mathbf{h}_j^{(l)}\right), \tag{8}$$

1. $\mathbf{h}_i^l \in \mathbb{R}^{d_l}$ is the hidden representation vector of node $i \in \mathcal{V}$ with dimensionality d_l after the lth message passing step (or 'layer')
2. σ is a pointwise non-linearity, e.g., ReLU activation function
3. $\mathbf{W}_0^{(l)}, \mathbf{W}_1^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ are learnable parameter matrices
4. $c_{i,j} = \frac{1}{\sqrt{D_{i,i}} \times \sqrt{D_{j,j}}}$ is a normalization constant

First-Order Model

We can write the GCN message passing update more compactly in matrix form:

$$\mathbf{H}^{(l+1)} = \sigma\left(\mathbf{H}^{(l)}\mathbf{W}_0^{(l)} + \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}_1^{(l)}\right), \tag{9}$$

where $\mathbf{H}^{(0)} = \mathbf{X}$.

First-Order Approximation

If $K = 1$, Eq. (7) can be simplified as

$$\begin{aligned} g_{\boldsymbol{\theta}'} * \mathbf{x} &\approx \theta'_0 \mathbf{x} + \theta'_1 (\mathbf{L} - \mathbf{I}_N) \mathbf{x} \\ &= \theta'_0 \mathbf{x} - \theta'_1 \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{x}. \end{aligned} \tag{10}$$

where $\lambda_{\max} \approx 2$.

Restricting the (linear) filtering operation to the first-order neighborhood in this way can reduce overfitting by using fewer free parameters per filtering operation, and hence might prove useful in contexts where only few labels are available or where a model is required to generalize in an inductive setting to unseen parts of a graph.

Generalization of the First-Order Model

This definition can be generalized to a signal $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{in}}}$ with d_{in} input channels and d_{out} filters or feature maps as follows:

$$\mathbf{Z} = \mathbf{X}\mathbf{\Theta}_0 - \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}\mathbf{X}\mathbf{\Theta}_1, \tag{11}$$

where $\mathbf{\Theta}_0$, $\mathbf{\Theta}_1 \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ are matrices of filter parameters, and $\mathbf{Z} \in \mathbb{R}^{N \times d_{\text{out}}}$ is the convolved signal matrix.

Single Parameter Model

In semi-supervised learning, overfitting to a small set of labeled nodes can often be an issue. This can be addressed by only using a single parameter matrix $\mathbf{W}^{(l)}$ per layer:

$$\mathbf{H}^{(l+1)} = \sigma\left(\left(\mathbf{I}_N + \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}\right)\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right). \tag{12}$$

We find that it can have positive impact on training performance to renormalize the adjacency matrix with added self-connections as
$\mathbf{I}_N + \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}} \rightarrow \tilde{\mathbf{D}}^{-\frac{1}{2}}\left(\mathbf{A} + \mathbf{I}_N\right)\tilde{\mathbf{D}}^{-\frac{1}{2}}$

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right). \tag{13}$$

Semi-Supervised Node Classification

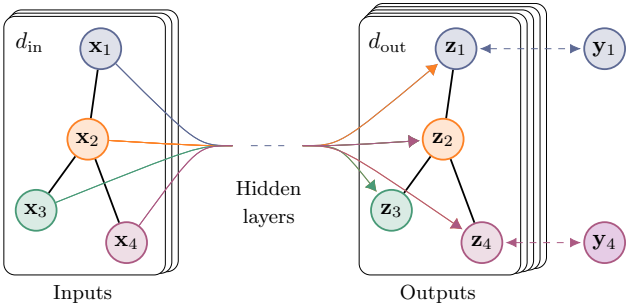


Figure 3.1: Schematic depiction of a multi-layer GCN for semi-supervised classification with d_{in} input channels and d_{out} feature maps in the output layer. The graph structure (edges are shown as black lines) is shared over layers. $\mathbf{x}_i$ are input features, $\mathbf{z}_i$ are node-wise predictions, and node labels are denoted by $\mathbf{y}_i$.

Semi-Supervised Node Classification [2]

We consider a two-layer GCN for semi-supervised node classification on a graph with a symmetric binary adjacency matrix $\mathbf{A}$:

1. Pre-processing: renormalized adjacency matrix $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$

Semi-Supervised Node Classification [2]

We consider a two-layer GCN for semi-supervised node classification on a graph with a symmetric binary adjacency matrix $\mathbf{A}$:

1. Pre-processing: renormalized adjacency matrix $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$
2. Our forward model then takes the simple form:

$$\mathbf{Z} = f(\mathbf{X}, \mathbf{A}) = \text{softmax}\left(\hat{\mathbf{A}}\text{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(0)}\right)\mathbf{W}^{(1)}\right). \tag{14}$$

Semi-Supervised Node Classification [2]

We consider a two-layer GCN for semi-supervised node classification on a graph with a symmetric binary adjacency matrix $\mathbf{A}$:

1. Pre-processing: renormalized adjacency matrix $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$
2. Our forward model then takes the simple form:

$$\mathbf{Z} = f(\mathbf{X}, \mathbf{A}) = \text{softmax}\left(\hat{\mathbf{A}}\,\text{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(0)}\right)\mathbf{W}^{(1)}\right). \tag{14}$$

3. We optimize the GCN for the task of semi-supervised node classification using the following cross-entropy loss on all labeled nodes:

$$\mathcal{L}_{\text{sup}} = -\sum_{l \in \mathcal{Y}_L} \sum_{f=1}^{d_{\text{out}}} Y_{l,f} \ln Z_{l,f}. \tag{15}$$

Outline

1. Clarification
2. Background
3. Methods
4. Experiments
5. Discussion
6. References

Datasets

1. Citation Networks: In the citation network datasets [3], nodes are documents and edges are citation links.

Table 3.1: Dataset statistics, as reported in Yang et al. (2016).

Dataset	Nodes	Edges	Classes	Features	Label rate
Citeseer	3,327	4,732	6	3,703	0.036
Cora	2,708	5,429	7	1,433	0.052
Pubmed	19,717	44,338	3	500	0.003

Datasets

1. Citation Networks: In the citation network datasets [3], nodes are documents and edges are citation links.

Table 3.1: Dataset statistics, as reported in Yang et al. (2016).

Dataset	Nodes	Edges	Classes	Features	Label rate
Citeseer	3,327	4,732	6	3,703	0.036
Cora	2,708	5,429	7	1,433	0.052
Pubmed	19,717	44,338	3	500	0.003

2. Random Graphs

Implementation Details

We make use of TensorFlow (Abadi et al., 2016) for an efficient GPU-based implementation of Eq. 3.12 (Eq. (14)) using sparse-dense matrix multiplications. The implementation is available under https://github.com/tkipf/gcn.

1. Test set: 1000 labeled examples
2. Validation set: 500 labeled examples
3. Dataset splits: same as Yang et al. [3].
4. Training iterations: 3000
5. Learning rate: 0.001 with Adam

Ref [3]

For each dataset, we split all instances into three parts, labeled data, unlabeled data, and test data. Inductive methods are trained on the labeled and unlabeled data, and tested on the test data. Transductive methods, on the other hand, are trained on the labeled, unlabeled data, and test data without labels.

Prediction Accuracy

Table 3.2: Summary of results in terms of classification accuracy in percent. See text for details.

Method	Citeseer	Cora	Pubmed
ManiReg (Belkin et al., 2006)	60.1	59.5	70.7
SemiEmb (Weston et al., 2012)	59.6	59.0	71.1
LP (Zhu et al., 2003)	45.3	68.0	63.0
DeepWalk (Perozzi et al., 2014)	43.2	67.2	65.3
Planetoid (Yang et al., 2016)	64.7 (26s)	75.7 (13s)	77.2 (25s)
GCN (Our method)	**70.3** (7s)	**81.5** (4s)	**79.0** (38s)
GCN (Random splits)	67.9 ± 0.5	80.1 ± 0.5	78.9 ± 0.7

Propagation Model Evaluation

Table 3.3: Propagation model evaluation. See text for details.

Description	**Propagation model**	**Citeseer**	**Cora**	**Pubmed**
Chebyshev ($K = 3$)	$\sum_{k=0}^{K} T_k(\tilde{\mathbf{L}})\mathbf{X}\mathbf{W}_k$	69.8	79.5	74.4
Chebyshev ($K = 2$)		69.6	81.2	73.8
1st-order	$\mathbf{X}\mathbf{W}_0 + \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}\mathbf{X}\mathbf{W}_1$	68.3	80.0	77.5
Single parameter	$(\mathbf{I}_N + \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}})\mathbf{X}\mathbf{W}$	69.3	79.2	77.4
Renormalized	$\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{X}\mathbf{W}$	**70.3**	**81.5**	**79.0**
1st-order term only	$\mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}\mathbf{X}\mathbf{W}$	68.7	80.5	77.8
MLP	$\mathbf{X}\mathbf{W}$	46.5	55.1	71.4

Training Time per Epoch

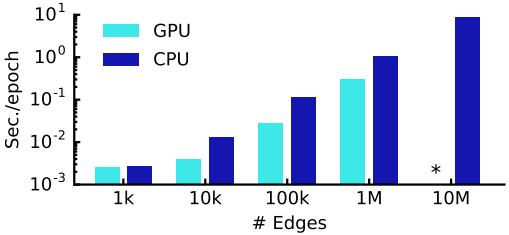


Figure 3.2: Experimental measurements of wall-clock time per epoch for simulated random graphs for a GPU and a CPU-only implementation. The GPU ran out of memory for the graph with 10 million edges.

Outline

1. Clarification
2. Background
3. Methods
4. Experiments
5. Discussion
6. References

Highlights & Limitations

Highlights

1. A single set of hyperparameters
2. Efficient

Limitations & Future Work

1. Memory requirement (for large and densely connected graph datasets)
2. Fixed Weighing of Neighboring Nodes
3. Directed Edges and Edge Features

- T. Kipf *et al.*, *Deep learning with graph-structured representations*, 2020.
- T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- Z. Yang, W. Cohen, and R. Salakhudinov, "Revisiting semi-supervised learning with graph embeddings," in *International conference on machine learning*, 2016, pp. 40–48.